HDB Resale – ETL / ELT

Technical Setup & Execution Guide

Databricks (Sandbox) | Medallion Architecture (DWH) | Snowflake Schema |Python | PySpark | Delta Tables

Table of Contents

1. Introduction 1
1.1 Notebook Overview 2
1.2 What the Notebook does? 2
1.3 Tech Stack: 2
1.4 Important Note: 2
2. Prerequisites 3
2.1 Platform 3
2.2 Source Data 3
2.3 Notebook Location 3
3.Configuration (00_nb_configs) 4
3.1 Settings You May Need to Change 4
3.2 Cloud / Non-Databricks Environments 4
3.2.1 Important - Environment Note: 4
4. Running the Notebooks 4
4.1 Notebook Execution Order 4
4.2 Notebook Orchestration 5
5. Outputs 5
5.1 Bronze Layer 5
5.2 Silver Layer 5
5.3 Gold Layer - Reporting Tables 6
6. Troubleshooting 6
7. Data Quality Checks (Summary) 7
8. Appendix 7
8.1 Hardcoded Configuration 7

1. Introduction

This document is a step-by-step guide for setting up and running the HDB Resale Notebook. It is intended for anyone who needs to execute the pipeline - including those with limited technical background.

The notebooks processes Singapore HDB resale transaction data from 2012 to 2016 across five Databricks notebooks, applying data quality checks, transformations, and dimensional modelling before producing reporting-ready tables

1.1 Notebook Overview

No.	Notebook	Layer	Source	Destination
00	configs	-	-	Constants & helpers (no table)
01	raw	Bronze	Raw HDB dataset (input file)	bronze.raw_hdb
02	cleaned	Bronze	bronze.raw_hdb	bronze.cleaned_hdb, bronze.failed_hdb
03	transformed	Silver	bronze.cleaned_hdb	silver.transformed_hdb, silver.hashed_hdb
04	reporting	Gold	silver.hashed_hdb	gold.dim_date, gold.dim_location, gold.dim_flat, gold.dim_storey, gold.dim_lease, gold.fact_resale, gold.rpt_price_by_town_flattype, gold.rpt_lease_vs_price, gold.rpt_storey_price_premium

1.2 What the Notebook does?

- Ingests raw CSV files from a source folder in Databricks FileStore.
- Cleans the data and removes duplicates, validates key fields, and flags price anomalies.
- Transforms records and builds a unique resale identifier and SHA-256 hash per transaction.
- Produces a Snowflake Schema: dimension tables, a fact table, and 3 analytical reports.

1.3 Tech Stack:

The table below describes the core technology used in this project.

Cloud Platform	Details	DWH Architecture	Dataset Scope
Databricks (Sandbox Workspace)	Python, PySpark	Medallion (Bronze / Silver / Gold layers)	Jan 2012 to Dec 2016

1.4 Important Note: As a quick achievement to develop the notebooks within the timeline, the Databricks sandbox environment has been utilized. However, the notebooks are compatible with

and can be adapted to different environments by making a few adjustments in **00_nb_Configs, as mentioned in Section 3.2 below.**

2. Prerequisites

Before running the notebooks, make sure all the following are in place.

2.1 Platform

Requirement	Details
Databricks workspace	Access to a Databricks workspace - sandbox or cloud (AWS, Azure, GCP).
Cluster	A running cluster with Databricks Runtime 11.x or later (includes Spark 3.x and Python 3.x).
Permissions	Ability to create schemas/tables in the metastore and read/write files in DBFS.

2.2 Source Data

Expects one or more HDB resale CSV files uploaded to Databricks FileStore.

- Expected location: dbfs:/FileStore/hdb_datasets/
- Each CSV must have a header row with at minimum: month, town, flat_type, block, street_name, storey_range, floor_area_sqm, flat_model, lease_commence_date, resale_price.
- Files from different years may have different schemas (e.g. remaining_lease column added later). This is handled automatically.
- Uploading CSV files to DBFS,
    - In your Databricks workspace, go to DBFS > FileStore.
    - Create a folder named test (or match the SOURCE_PATH in 00_nb_configs).
    - Upload all HDB CSV files into that folder.

2.3 Notebook Location

All five notebooks must be in the same folder in the Databricks workspace. Notebook 01 through 04 each run notebook 00 (configs) using a relative %run command, so the files must be co-located.

- Recommended workspace path: /Workspace/Users/<your-email>/hdb/
- Upload or import all five notebooks into that folder.

3.Configuration (00_nb_configs)

All settings live in a single file: 00_nb_configs. You only ever need to edit this one file. All other notebooks inherit its values automatically.

3.1 Settings You May Need to Change

Setting	What it controls
SOURCE_PATH	Folder path where the HDB CSV files are stored in DBFS. Default: dbfs:/FileStore/hdb_datasets/
DATE_START / DATE_END	Date range filter for transactions. Default: 2012-01-01 to 2016-12-31
BRONZE_SCHEMA / SILVER_SCHEMA / GOLD_SCHEMA	Schema (database) names used when saving tables. Default: bronze, silver, gold
HDB_LEASE	Standard HDB lease duration in years. Default: 99
VALID_FLAT_TYPES	Whitelist of accepted flat type values. Any record not in this list is rejected.

3.2 Cloud / Non-Databricks Environments

3.2.1 Important - Environment Note:

These notebooks were developed in the Databricks sandbox environment as a quick achievement. They will also work on other cloud Databricks deployments (AWS, Azure, GCP) with no changes needed.

If you are running outside Databricks (e.g. a local Spark setup or a different cloud platform), the following tweaks are needed in 00_nb_configs:

- Create the SparkSession manually if one is not provided by the environment.
- Comment out the catalog usage lines after the Databrick – Catalog markdown cell.
- Comment out the schema creation line after the Ouput Delta Table Creation markdown.
- Replace the DBFS path in SOURCE_PATH with a local or cloud storage path.
- Replace saveAsTable(...) in write_layer() with df.write.parquet(<local-path>) or equivalent.
- Replace spark.read.table(...) with spark.read.parquet(<path>) in notebooks 02, 03, and 04.

4. Running the Notebooks

4.1 Notebook Execution Order

The notebooks must be run in sequence. Each notebook depends on the output of the one before it.

1. Open **00_nb_configs** and verify the settings (especially SOURCE_PATH and schema names).
2. Run **01_nb_bronze_raw_ingestion** - loads CSV files and writes the raw table.
3. Run **02_nb_bronze_cleansed** - validates and cleans the data, writes Cleaned and Failed tables.
4. Run **03_nb_silver_transformed** - builds identifiers and hashes, writes Transformed and Hashed tables.
5. Run **04_nb_gold_reporting** - builds the Snowflake schema and report tables.

Note: Each notebook includes markdown documentation explaining its purpose, transformations performed, and the business logic implemented.

4.2 Notebook Orchestration

To simplify execution and ensure the notebooks run in the correct sequence, a master orchestration notebook is provided. The notebook **99_nb_master_orchestration** coordinates the execution of the notebooks by sequentially triggering the required notebooks using `%run` commands. This eliminates the need to manually execute each notebook and ensures the pipeline runs consistently in the defined order.

5. Outputs

After a successful run, the following tables will be available in Databricks catalog.

5.1 Bronze Layer

Table	Description
bronze.raw_hdb	All raw records from source CSVs, filtered to the defined date range. No cleaning applied.
bronze.cleaned_hdb	Records that passed all data quality checks (deduplication, validation, price anomaly).
bronze.failed_hdb	Rejected records with a failed_reason column explaining why each was excluded.

5.2 Silver Layer

Table	Description
silver.transformed_hdb	Cleaned records enriched with the resale_identifier composite key.
silver.hashed_hdb	Transformed records with an added SHA-256 hashed_identifier column.

5.3 Gold Layer - Reporting Tables

Table	Description
gold.dim_date	Date dimension: year, quarter, month derived from transaction month.
gold.dim_location	Location dimension: town, street name, block.
gold.dim_flat	Flat dimension: flat type, model, floor area.
gold.dim_storey	Storey dimension: storey band with numeric low/high floor values.
gold.dim_lease	Lease dimension: lease commence date and recomputed remaining lease.
gold.fact_resale	Central fact table: one row per transaction with foreign keys to all dimensions.
gold.rpt_price_by_town_flattype	Average/min/max resale price by town, flat type, and year.
gold.rpt_lease_vs_price	Average resale price by remaining lease band (10-year buckets).
gold.rpt_storey_price_premium	Average resale price per storey band, ordered by floor level.

6. Troubleshooting

Issue	What to do
No records loaded in 01_nb_bronze_raw_ingestion	Check that SOURCE_PATH in configs points to the correct DBFS folder and that CSV files are present.
Table not found error in 02 / 03 / 04	Make sure the previous notebook ran successfully and that the bronze/silver/gold schemas exist.
%run error: notebook not found	Confirm all five notebooks are in the same workspace folder and the %run path matches the file name exactly.
Schema does not exist error	Run the CREATE SCHEMA statements from Section 2.3 before executing the notebooks.
Cluster not attached	Select a running cluster from the dropdown at the top of the notebook before running.
All records fail validation	Check that source CSV column names match what the notebooks expects (month, town, flat_type, etc).

7. Data Quality Checks (Summary)

This applies 3 layers of quality control in notebook 02. Records that fail any check are saved to bronze.failed_hdb with a reason.

Check	Rule applied
Deduplication	Duplicate rows (same composite key, different price) are removed. The record with the highest price is kept.
Field validation	month must not be null or in the future. town must be uppercase letters. flat_type must be in the approved list. storey_range must follow the NN-to-NN format.
Price anomaly detection	Records with a Z-Score greater than 3 within their town + flat_type group are flagged as anomalies and excluded.

8. Appendix

8.1 Hardcoded Configuration

The following parameters are currently hardcoded in the notebook configuration section. These values are used for filtering and business logic during data processing.

If requirements change in the future, these parameters can be updated in the configuration cell without modifying the logic.

Date Range Filter

Parameter	Value
DATE_START	2012-01-01
DATE_END	2016-12-31

Business Logic Constants

Parameter	Value
HDB_LEASE	99
VALID_FLAT_TYPES	1 ROOM, 2 ROOM, 3 ROOM, 4 ROOM, 5 ROOM, EXECUTIVE, MULTI-GENERATION.

— End of Document —